

Foot Ulcer Segmentation V.3



Notebook Input Output Logs Comments (0) Settings

In [1]:

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
```

Version 1 of 1



Runtime

▶ 1h 18m 19s · GPU T4 x2

Input

DATASETS

📁 Foot Ulcer Segmentation Dataset

Tags

GPU



Language

Python

In [2]:

```
!pip install segmentation-models-pytorch albumentations -q
```

154.8/154.8 kB 4.2 MB/s et

a 0:00:00

In [3]:

```
import kagglehub
path = kagglehub.dataset_download("sarayutsupsin/foot-ulcer-segmentation-dataset")
print("Path:", path)
```

Path: /kaggle/input/datasets/sarayutsupsin/foot-ulcer-segmentation-dataset

In [4]:

```
import os
for root, dirs, files in os.walk(path):
    level = root.replace(path, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}{os.path.basename(root)}/')
    if level < 3:
        for f in files[:3]:
            print(f'{indent} {f}')
```

```
foot-ulcer-segmentation-dataset/
dataset-wound-segmentation-Foot_Ulcer/
  validation/
    labels/
    images/
  test/
    images/
  train/
    labels/
    images/
```

In [5]:

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import albumentations as A
from albumentations.pytorch import ToTensorV2

import segmentation_models_pytorch as smp
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR

class Config:
    DATA_ROOT      = os.path.join(path, "dataset-wound-segmentation-Foot_Ulcer")
    SAVE_DIR        = "./checkpoints"
    IMAGE_SIZE      = 512
    BATCH_SIZE      = 8
    NUM_EPOCHS      = 50
    LR              = 3e-4
    THRESHOLD       = 0.5
    ENCODER         = "efficientnet-b4"
```

```
from torch.utils.data import Dataset, DataLoader
import albumentations as A
from albumentations.pytorch import ToTensorV2

import segmentation_models_pytorch as smp
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR
```

```
class Config:
    DATA_ROOT      = os.path.join(path, "dataset-wound-segmentation-Foot_Ulcer")
    SAVE_DIR        = "./checkpoints"
    IMAGE_SIZE      = 512
    BATCH_SIZE      = 8
    NUM_EPOCHS      = 50
    LR              = 3e-4
    THRESHOLD       = 0.5
    ENCODER         = "efficientnet-b4"
    ENCODER_WEIGHTS = "imagenet"
    DEVICE          = "cuda" if torch.cuda.is_available() else "cpu"
```

```
cfg = Config()
os.makedirs(cfg.SAVE_DIR, exist_ok=True)
print(f"Using device: {cfg.DEVICE}")
print(f"DATA_ROOT: {cfg.DATA_ROOT}")
```

Using device: cuda

DATA_ROOT: /kaggle/input/datasets/sarayutsupsin/foot-ulcer-segmentation-dataset/dataset-wound-segmentation-Foot_Ulcer

Dataset + Augmentation:

In [6]:

```
class WoundDataset(Dataset):
    def __init__(self, images_dir, labels_dir=None, transform=None):
        self.images_dir = Path(images_dir)
        self.labels_dir = Path(labels_dir) if labels_dir else None
        self.transform = transform
        self.image_files = sorted([
            f for f in os.listdir(images_dir)
            if f.lower().endswith((".png", ".jpg", ".jpeg"))
        ])

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        fname = self.image_files[idx]
        img_path = self.images_dir / fname
        image = cv2.imread(str(img_path))
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        if self.labels_dir is not None:
            label_path = self.labels_dir / fname
            if not label_path.exists():
                stem = Path(fname).stem
                for ext in [".png", ".jpg", ".jpeg"]:
                    candidate = self.labels_dir / (stem + ext)
                    if candidate.exists():
                        label_path = candidate
```

```
def __getitem__(self, idx):
    fname = self.image_files[idx]
    img_path = self.images_dir / fname
    image = cv2.imread(str(img_path))
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    if self.labels_dir is not None:
        label_path = self.labels_dir / fname
        if not label_path.exists():
            stem = Path(fname).stem
            for ext in [".png", ".jpg", ".jpeg"]:
                candidate = self.labels_dir / (stem + ext)
                if candidate.exists():
                    label_path = candidate
                    break

    mask = cv2.imread(str(label_path), cv2.IMREAD_GRAYSCALE)
    mask = (mask > 127).astype(np.float32)

    if self.transform:
        augmented = self.transform(image=image, mask=mask)
        image = augmented["image"]
        mask = augmented["mask"].unsqueeze(0)
        return image, mask
    else:
        if self.transform:
            augmented = self.transform(image=image)
            image = augmented["image"]
        return image, fname
```

```

        if self.transform:
            augmented = self.transform(image=image)
            image = augmented["image"]
        return image, fname

def get_transforms(phase):
    mean = [0.485, 0.456, 0.406]
    std = [0.229, 0.224, 0.225]
    if phase == "train":
        return A.Compose([
            A.Resize(cfg.IMAGE_SIZE, cfg.IMAGE_SIZE),
            A.HorizontalFlip(p=0.5),
            A.VerticalFlip(p=0.3),
            A.RandomRotate90(p=0.3),
            A.ShiftScaleRotate(shift_limit=0.05, scale_limit=0.1, rotate_limit=15, p=0.4),
            A.RandomBrightnessContrast(p=0.3),
            A.HueSaturationValue(p=0.2),
            A.GaussNoise(p=0.2),
            A.Normalize(mean=mean, std=std),
            ToTensorV2(),
        ])
    else:
        return A.Compose([
            A.Resize(cfg.IMAGE_SIZE, cfg.IMAGE_SIZE),
            A.Normalize(mean=mean, std=std),
            ToTensorV2(),
        ])

print("Dataset class ready ✓")

```

Dataset class ready ✓

เชื่อว่าโหลด dataset ได้ถูกต้อง:

```

In [7]: train_ds = WoundDataset(
        images_dir = os.path.join(cfg.DATA_ROOT, "train/images"),
        labels_dir = os.path.join(cfg.DATA_ROOT, "train/labels"),
        transform = get_transforms("train"),
    )
    val_ds = WoundDataset(
        images_dir = os.path.join(cfg.DATA_ROOT, "validation/images"),
        labels_dir = os.path.join(cfg.DATA_ROOT, "validation/labels"),
        transform = get_transforms("val"),
    )

    print(f"Train: {len(train_ds)} images")
    print(f"Val: {len(val_ds)} images")

    # เช็ค shape ของ 1 sample
    img, mask = train_ds[0]
    print(f"Image shape: {img.shape}")
    print(f"Mask shape: {mask.shape}")

```

```

Train: 810 images
Val: 200 images
Image shape: torch.Size([3, 512, 512])
Mask shape: torch.Size([1, 512, 512])

```

```

/usr/local/lib/python3.12/dist-packages/albumentations/core/validation.py:114: UserWarning: Shift
ScaleRotate is a special case of Affine transform. Please use Affine transform instead.

```

In [8]:

```
def build_model():
    model = smp.Unet(
        encoder_name      = cfg.ENCODER,
        encoder_weights   = cfg.ENCODER_WEIGHTS,
        in_channels       = 3,
        classes           = 1,
        activation         = None,
    )
    return model.to(cfg.DEVICE)

class DiceBCELoss(nn.Module):
    def __init__(self, smooth=1e-6):
        super().__init__()
        self.bce = nn.BCEWithLogitsLoss()
        self.smooth = smooth

    def forward(self, logits, targets):
        bce_loss = self.bce(logits, targets)
        probs = torch.sigmoid(logits)
        inter = (probs * targets).sum(dim=(2, 3))
        union = probs.sum(dim=(2, 3)) + targets.sum(dim=(2, 3))
        dice_loss = 1 - (2 * inter + self.smooth) / (union + self.smooth)
        return bce_loss + dice_loss.mean()

def iou_score(preds, targets, smooth=1e-6):
    preds = (torch.sigmoid(preds) > cfg.THRESHOLD).float()
    inter = (preds * targets).sum(dim=(2, 3))
    union = preds.sum(dim=(2, 3)) + targets.sum(dim=(2, 3)) - inter
    return ((inter + smooth) / (union + smooth)).mean().item()
```

```
def iou_score(preds, targets, smooth=1e-6):
    preds = (torch.sigmoid(preds) > cfg.THRESHOLD).float()
    inter = (preds * targets).sum(dim=(2, 3))
    union = preds.sum(dim=(2, 3)) + targets.sum(dim=(2, 3)) - inter
    return ((inter + smooth) / (union + smooth)).mean().item()

def dice_score(preds, targets, smooth=1e-6):
    preds = (torch.sigmoid(preds) > cfg.THRESHOLD).float()
    inter = (preds * targets).sum(dim=(2, 3))
    union = preds.sum(dim=(2, 3)) + targets.sum(dim=(2, 3))
    return ((2 * inter + smooth) / (union + smooth)).mean().item()

print("Model + Loss + Metrics ready ✓")
```

Model + Loss + Metrics ready ✓

Train/Eval functions:

```
In [9]: def train_one_epoch(model, loader, optimizer, criterion):
        model.train()
        total_loss, total_iou, total_dice = 0, 0, 0
        for images, masks in loader:
            images = images.to(cfg.DEVICE)
            masks = masks.to(cfg.DEVICE)
            optimizer.zero_grad()
            logits = model(images)
            loss = criterion(logits, masks)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            total_iou += iou_score(logits.detach(), masks)
            total_dice += dice_score(logits.detach(), masks)
        n = len(loader)
        return total_loss/n, total_iou/n, total_dice/n

    @torch.no_grad()
    def evaluate(model, loader, criterion):
        model.eval()
        total_loss, total_iou, total_dice = 0, 0, 0
        for images, masks in loader:
            images = images.to(cfg.DEVICE)
            masks = masks.to(cfg.DEVICE)
            logits = model(images)
            loss = criterion(logits, masks)
            total_loss += loss.item()
```

```
@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, total_iou, total_dice = 0, 0, 0
    for images, masks in loader:
        images = images.to(cfg.DEVICE)
        masks = masks.to(cfg.DEVICE)
        logits = model(images)
        loss = criterion(logits, masks)
        total_loss += loss.item()
        total_iou += iou_score(logits, masks)
        total_dice += dice_score(logits, masks)
    n = len(loader)
    return total_loss/n, total_iou/n, total_dice/n

print("Train/Eval functions ready ✓")
```

Train/Eval functions ready ✓

Train

```
In [10]: train_loader = DataLoader(train_ds, batch_size=cfg.BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
val_loader = DataLoader(val_ds, batch_size=cfg.BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

model = build_model() # สร้าง U-Net + EfficientNet-B4
criterion = DiceBCELoss() # ฟังก์ชันคำนวณ error
optimizer = AdamW(model.parameters(), lr=cfg.LR, weight_decay=1e-4) # ตัวปรับ weight ของ model
scheduler = CosineAnnealingLR(optimizer, T_max=cfg.NUM_EPOCHS, eta_min=1e-6) # ค่อยๆ ลด learning rate
ตาม epoch

best_dice = 0.0
history = {"train_loss": [], "val_loss": [], "train_iou": [], "val_iou": [], "train_dice": [], "val_dice": []}

for epoch in range(1, cfg.NUM_EPOCHS + 1):
    train_loss, train_iou, train_dice = train_one_epoch(model, train_loader, optimizer, criterion)
    val_loss, val_iou, val_dice = evaluate(model, val_loader, criterion)
    scheduler.step()

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_iou"].append(train_iou)
    history["val_iou"].append(val_iou)
    history["train_dice"].append(train_dice)
    history["val_dice"].append(val_dice)

    print(f"Epoch [{epoch:02d}/{cfg.NUM_EPOCHS}] "
          f"Loss: {train_loss:.4f}/{val_loss:.4f} | "
          f"IoU: {train_iou:.4f}/{val_iou:.4f} | "
```

```

print(f"Epoch [{epoch:02d}/{cfg.NUM_EPOCHS}] "
      f"Loss: {train_loss:.4f}/{val_loss:.4f} | "
      f"IoU: {train_iou:.4f}/{val_iou:.4f} | "
      f"Dice: {train_dice:.4f}/{val_dice:.4f}")

if val_dice > best_dice:
    best_dice = val_dice
    torch.save(model.state_dict(), os.path.join(cfg.SAVE_DIR, "best_model.pth")) # เซฟเฉพาะตอนที่ v
al Dice ดีขึ้นเท่านั้น
    print(f" ✓ Saved best model (Dice: {best_dice:.4f})")

print(f"\nDone! Best Val Dice: {best_dice:.4f}")

```

```

Epoch [01/50] Loss: 1.4579/1.1866 | IoU: 0.2616/0.5750 | Dice: 0.3444/0.6913
  ✓ Saved best model (Dice: 0.6913)
Epoch [02/50] Loss: 1.0367/0.9203 | IoU: 0.5330/0.6296 | Dice: 0.6578/0.7367
  ✓ Saved best model (Dice: 0.7367)
Epoch [03/50] Loss: 0.8052/0.6981 | IoU: 0.5977/0.6912 | Dice: 0.7112/0.7862
  ✓ Saved best model (Dice: 0.7862)
Epoch [04/50] Loss: 0.5838/0.5327 | IoU: 0.6358/0.6167 | Dice: 0.7414/0.7393
Epoch [05/50] Loss: 0.3992/0.3242 | IoU: 0.6749/0.7417 | Dice: 0.7733/0.8243
  ✓ Saved best model (Dice: 0.8243)
Epoch [06/50] Loss: 0.2946/0.2515 | IoU: 0.7049/0.7223 | Dice: 0.8002/0.8121
Epoch [07/50] Loss: 0.2476/0.2498 | IoU: 0.7200/0.7206 | Dice: 0.8113/0.8077
Epoch [08/50] Loss: 0.2254/0.2428 | IoU: 0.7325/0.7077 | Dice: 0.8231/0.7975
Epoch [09/50] Loss: 0.2109/0.1913 | IoU: 0.7404/0.7611 | Dice: 0.8243/0.8365
  ✓ Saved best model (Dice: 0.8365)
Epoch [10/50] Loss: 0.2075/0.1894 | IoU: 0.7302/0.7504 | Dice: 0.8250/0.8244

```

Epoch [08/50] Loss: 0.2254/0.2428 | IoU: 0.7325/0.7077 | Dice: 0.8231/0.7975
Epoch [09/50] Loss: 0.2109/0.1913 | IoU: 0.7404/0.7611 | Dice: 0.8243/0.8365
✓ Saved best model (Dice: 0.8365)
Epoch [10/50] Loss: 0.2075/0.1984 | IoU: 0.7393/0.7504 | Dice: 0.8258/0.8344
Epoch [11/50] Loss: 0.1963/0.1779 | IoU: 0.7537/0.7727 | Dice: 0.8384/0.8489
✓ Saved best model (Dice: 0.8489)
Epoch [12/50] Loss: 0.1916/0.1904 | IoU: 0.7574/0.7559 | Dice: 0.8398/0.8354
Epoch [13/50] Loss: 0.1950/0.1806 | IoU: 0.7484/0.7672 | Dice: 0.8330/0.8398
Epoch [14/50] Loss: 0.1834/0.1889 | IoU: 0.7580/0.7678 | Dice: 0.8413/0.8405
Epoch [15/50] Loss: 0.1770/0.1738 | IoU: 0.7660/0.7690 | Dice: 0.8462/0.8435
Epoch [16/50] Loss: 0.1720/0.1706 | IoU: 0.7706/0.7715 | Dice: 0.8495/0.8467
Epoch [17/50] Loss: 0.1681/0.1853 | IoU: 0.7724/0.7593 | Dice: 0.8515/0.8330
Epoch [18/50] Loss: 0.1676/0.1704 | IoU: 0.7779/0.7735 | Dice: 0.8550/0.8461
Epoch [19/50] Loss: 0.1663/0.1688 | IoU: 0.7769/0.7754 | Dice: 0.8531/0.8470
Epoch [20/50] Loss: 0.1585/0.1628 | IoU: 0.7846/0.7803 | Dice: 0.8589/0.8519
✓ Saved best model (Dice: 0.8519)
Epoch [21/50] Loss: 0.1564/0.1671 | IoU: 0.7861/0.7759 | Dice: 0.8618/0.8479
Epoch [22/50] Loss: 0.1687/0.1658 | IoU: 0.7736/0.7772 | Dice: 0.8519/0.8487
Epoch [23/50] Loss: 0.1547/0.1640 | IoU: 0.7860/0.7789 | Dice: 0.8616/0.8506
Epoch [24/50] Loss: 0.1606/0.1638 | IoU: 0.7806/0.7794 | Dice: 0.8571/0.8512
Epoch [25/50] Loss: 0.1561/0.1598 | IoU: 0.7864/0.7838 | Dice: 0.8613/0.8551
✓ Saved best model (Dice: 0.8551)
Epoch [26/50] Loss: 0.1568/0.1597 | IoU: 0.7845/0.7859 | Dice: 0.8590/0.8548
Epoch [27/50] Loss: 0.1540/0.1603 | IoU: 0.7874/0.7838 | Dice: 0.8613/0.8541
Epoch [28/50] Loss: 0.1489/0.1610 | IoU: 0.7943/0.7835 | Dice: 0.8666/0.8531
Epoch [29/50] Loss: 0.1502/0.1596 | IoU: 0.7928/0.7840 | Dice: 0.8653/0.8542
Epoch [30/50] Loss: 0.1458/0.1596 | IoU: 0.7966/0.7835 | Dice: 0.8680/0.8540
Epoch [31/50] Loss: 0.1452/0.1609 | IoU: 0.8010/0.7820 | Dice: 0.8718/0.8525
Epoch [32/50] Loss: 0.1445/0.1579 | IoU: 0.7984/0.7854 | Dice: 0.8700/0.8558
✓ Saved best model (Dice: 0.8558)
Epoch [33/50] Loss: 0.1437/0.1624 | IoU: 0.8005/0.7803 | Dice: 0.8723/0.8512
Epoch [34/50] Loss: 0.1453/0.1601 | IoU: 0.8003/0.7835 | Dice: 0.8709/0.8536

```

Epoch [32/50] Loss: 0.1445/0.1579 | IoU: 0.7984/0.7854 | Dice: 0.8700/0.8558
  ✓ Saved best model (Dice: 0.8558)
Epoch [33/50] Loss: 0.1437/0.1624 | IoU: 0.8005/0.7803 | Dice: 0.8723/0.8512
Epoch [34/50] Loss: 0.1453/0.1601 | IoU: 0.8003/0.7835 | Dice: 0.8709/0.8536
Epoch [35/50] Loss: 0.1432/0.1582 | IoU: 0.8005/0.7861 | Dice: 0.8713/0.8554
Epoch [36/50] Loss: 0.1361/0.1564 | IoU: 0.8106/0.7874 | Dice: 0.8807/0.8576
  ✓ Saved best model (Dice: 0.8576)
Epoch [37/50] Loss: 0.1391/0.1563 | IoU: 0.8050/0.7882 | Dice: 0.8753/0.8575
Epoch [38/50] Loss: 0.1340/0.1560 | IoU: 0.8137/0.7882 | Dice: 0.8820/0.8578
  ✓ Saved best model (Dice: 0.8578)
Epoch [39/50] Loss: 0.1352/0.1588 | IoU: 0.8092/0.7844 | Dice: 0.8776/0.8548
Epoch [40/50] Loss: 0.1347/0.1577 | IoU: 0.8097/0.7856 | Dice: 0.8790/0.8553
Epoch [41/50] Loss: 0.1373/0.1569 | IoU: 0.8089/0.7860 | Dice: 0.8781/0.8561
Epoch [42/50] Loss: 0.1324/0.1547 | IoU: 0.8137/0.7882 | Dice: 0.8815/0.8582
  ✓ Saved best model (Dice: 0.8582)
Epoch [43/50] Loss: 0.1362/0.1565 | IoU: 0.8105/0.7867 | Dice: 0.8797/0.8566
Epoch [44/50] Loss: 0.1383/0.1568 | IoU: 0.8099/0.7874 | Dice: 0.8778/0.8565
Epoch [45/50] Loss: 0.1364/0.1555 | IoU: 0.8121/0.7879 | Dice: 0.8808/0.8575
Epoch [46/50] Loss: 0.1339/0.1567 | IoU: 0.8128/0.7867 | Dice: 0.8808/0.8565
Epoch [47/50] Loss: 0.1328/0.1570 | IoU: 0.8129/0.7863 | Dice: 0.8805/0.8559
Epoch [48/50] Loss: 0.1333/0.1567 | IoU: 0.8122/0.7867 | Dice: 0.8802/0.8563
Epoch [49/50] Loss: 0.1388/0.1567 | IoU: 0.8054/0.7865 | Dice: 0.8746/0.8564
Epoch [50/50] Loss: 0.1372/0.1569 | IoU: 0.8087/0.7863 | Dice: 0.8766/0.8562

```

Done! Best Val Dice: 0.8582

Plot training curves:

In [11]:

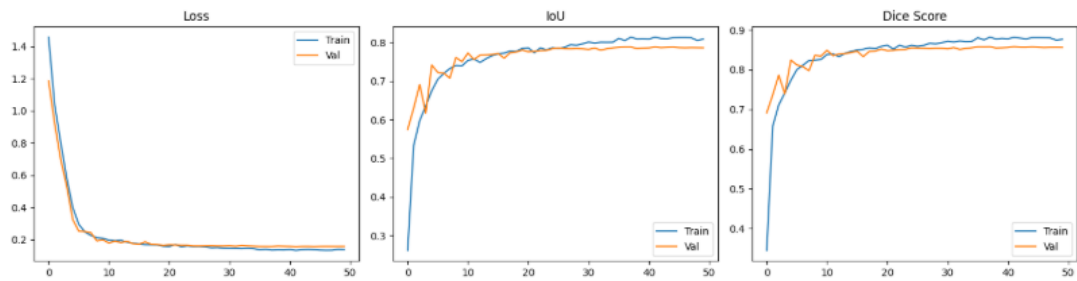
```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].plot(history["train_loss"], label="Train")
axes[0].plot(history["val_loss"], label="Val")
axes[0].set_title("Loss"); axes[0].legend()

axes[1].plot(history["train_iou"], label="Train")
axes[1].plot(history["val_iou"], label="Val")
axes[1].set_title("IoU"); axes[1].legend()

axes[2].plot(history["train_dice"], label="Train")
axes[2].plot(history["val_dice"], label="Val")
axes[2].set_title("Dice Score"); axes[2].legend()

plt.tight_layout()
plt.show()
```



prediction vs ground truth:

In [12]:

```
@torch.no_grad()
def predict_and_visualize(num_samples=10):
    model.eval()
    loader = DataLoader(val_ds, batch_size=1, shuffle=True)
    fig, axes = plt.subplots(num_samples, 3, figsize=(12, num_samples * 4))

    for i, (image, mask) in enumerate(loader):
        if i >= num_samples:
            break

        logit = model(image.to(cfg.DEVICE))
        pred = (torch.sigmoid(logit) > cfg.THRESHOLD).float().cpu()

        img_np = image[0].permute(1, 2, 0).numpy()
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        img_np = (img_np * std + mean).clip(0, 1)

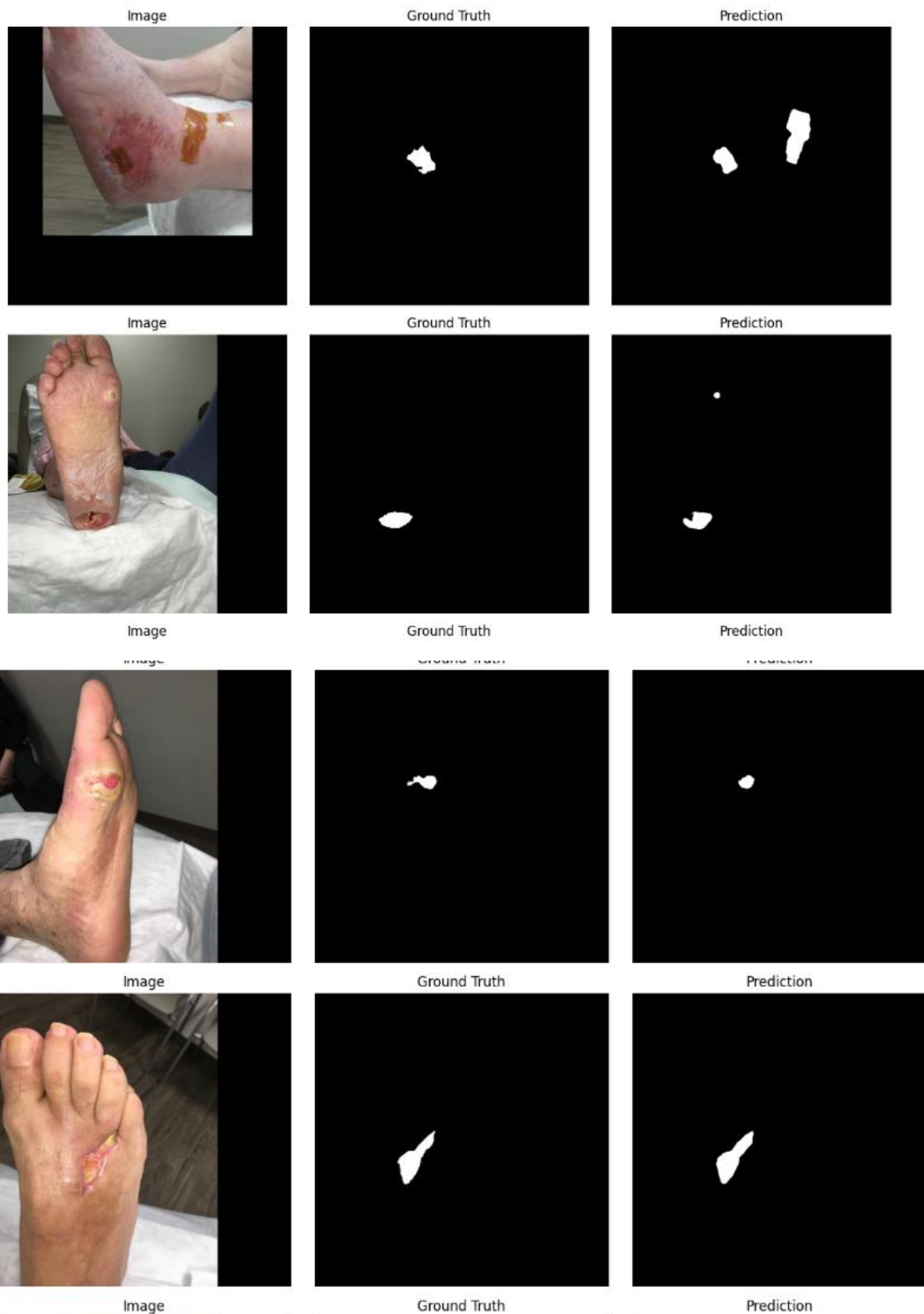
        axes[i][0].imshow(img_np); axes[i][0].set_title("Image"); axes
[i][0].axis("off")
        axes[i][1].imshow(mask[0,0], cmap="gray"); axes[i][1].set_title("Ground Truth"); axes
[i][1].axis("off")
        axes[i][2].imshow(pred[0,0], cmap="gray"); axes[i][2].set_title("Prediction"); axes
[i][2].axis("off")

        plt.tight_layout()
        plt.show()

predict_and_visualize()
```

```
plt.tight_layout()
plt.show()

predict_and_visualize()
```



Test

In [13]:

```
@torch.no_grad()
def predict_test(num_samples=5):
    test_ds = WoundDataset(
        images_dir = os.path.join(cfg.DATA_ROOT, "test/images"),
        transform = get_transforms("val"),
    )
    loader = DataLoader(test_ds, batch_size=1, shuffle=True)

    fig, axes = plt.subplots(num_samples, 2, figsize=(8, num_samples * 4))

    for i, (image, fname) in enumerate(loader):
        if i >= num_samples:
            break

        logit = model(image.to(cfg.DEVICE))
        pred = (torch.sigmoid(logit) > cfg.THRESHOLD).float().cpu()

        img_np = image[0].permute(1, 2, 0).numpy()
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        img_np = (img_np * std + mean).clip(0, 1)

        axes[i][0].imshow(img_np); axes[i][0].set_title(f"Image: {fname[0]}"); axes[i][
0].axis("off")
        axes[i][1].imshow(pred[0,0], cmap="gray"); axes[i][1].set_title("Predicted Mask"); axes
[i][1].axis("off")
```

```
axes[i][0].imshow(img_np); axes[i][0].set_title(f"Image: {fname[0]}"); axes[i]
[0].axis("off")
axes[i][1].imshow(pred[0,0], cmap="gray"); axes[i][1].set_title("Predicted Mask"); axes
[i][1].axis("off")

plt.tight_layout()
plt.show()

predict_test()
```

Image: 1365.png



Predicted Mask

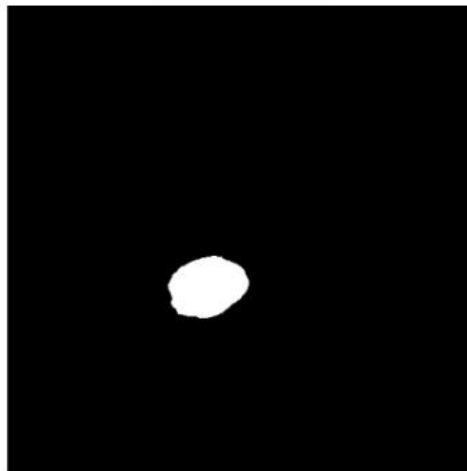


Image: 1271.png



Predicted Mask

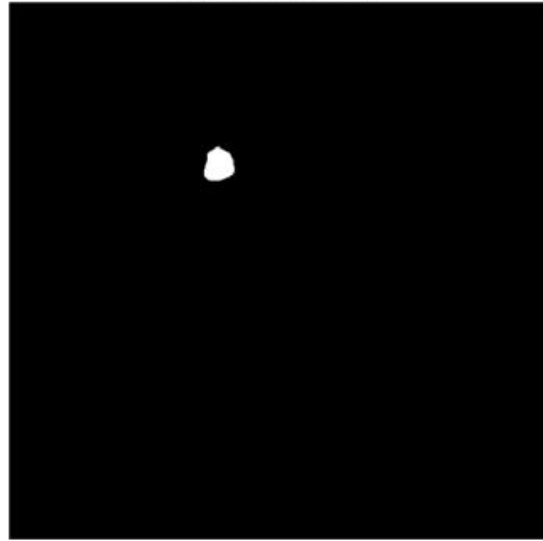
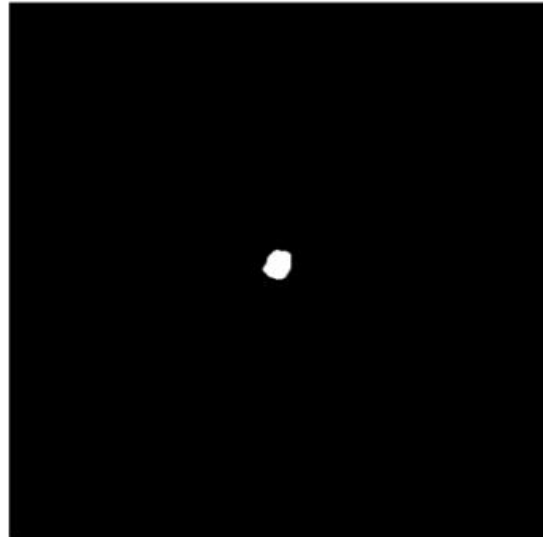


Image: 1283.png



Predicted Mask



In [14]:

```
@torch.no_grad()
def predict_test(num_samples=5, threshold=0.2): # ลด threshold จาก 0.5 → 0.3
    test_ds = WoundDataset(
        images_dir = os.path.join(cfg.DATA_ROOT, "test/images"),
        transform = get_transforms("val"),
    )
    loader = DataLoader(test_ds, batch_size=1, shuffle=True)

    fig, axes = plt.subplots(num_samples, 2, figsize=(8, num_samples * 4))

    for i, (image, fname) in enumerate(loader):
        if i >= num_samples:
            break

        logit = model(image.to(cfg.DEVICE))
        pred = (torch.sigmoid(logit) > threshold).float().cpu()

        img_np = image[0].permute(1, 2, 0).numpy()
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        img_np = (img_np * std + mean).clip(0, 1)

        axes[i][0].imshow(img_np); axes[i][0].set_title(f"Image: {fname[0]}"); axes
[i][0].axis("off")
        axes[i][1].imshow(pred[0,0], cmap="gray"); axes[i][1].set_title("Predicted Mask"); axes
[i][1].axis("off")

        plt.tight_layout()
        plt.show()

predict_test(threshold=0.3)
```

```
[i][1].axis("off")  
  
plt.tight_layout()  
plt.show()  
  
predict_test(threshold=0.3)
```

Image: 1165.png



Predicted Mask

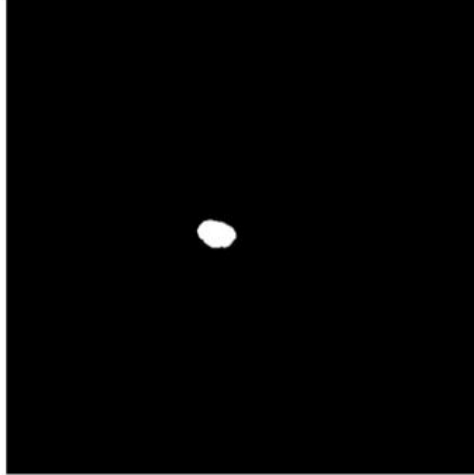
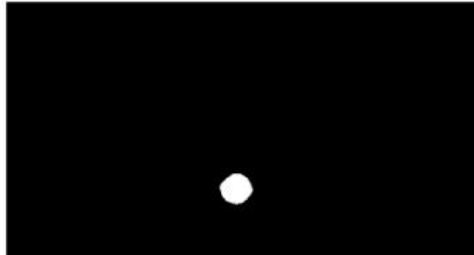


Image: 1144.png



Predicted Mask



In [15]:

```
@torch.no_grad()
def predict_test(num_samples=5, threshold=0.2): # ลด threshold จาก 0.5 → 0.3
    test_ds = WoundDataset(
        images_dir = os.path.join(cfg.DATA_ROOT, "test/images"),
        transform = get_transforms("val"),
    )
    loader = DataLoader(test_ds, batch_size=1, shuffle=True)

    fig, axes = plt.subplots(num_samples, 2, figsize=(8, num_samples * 4))

    for i, (image, fname) in enumerate(loader):
        if i >= num_samples:
            break

        logit = model(image.to(cfg.DEVICE))
        pred = (torch.sigmoid(logit) > threshold).float().cpu()

        img_np = image[0].permute(1, 2, 0).numpy()
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        img_np = (img_np * std + mean).clip(0, 1)

        axes[i][0].imshow(img_np); axes[i][0].set_title(f"Image: {fname[0]}"); axes
[i][0].axis("off")
        axes[i][1].imshow(pred[0,0], cmap="gray"); axes[i][1].set_title("Predicted Mask"); axes
[i][1].axis("off")

    plt.tight_layout()
    plt.show()
```

```
axes[i][1].imshow(pred[0,0], cmap="gray"); axes[i][1].set_title("Predicted Mask"); axes
[i][1].axis("off")

plt.tight_layout()
plt.show()

predict_test(threshold=0.3)
```

Image: 1074.png



Predicted Mask



Image: 1038.png



Predicted Mask

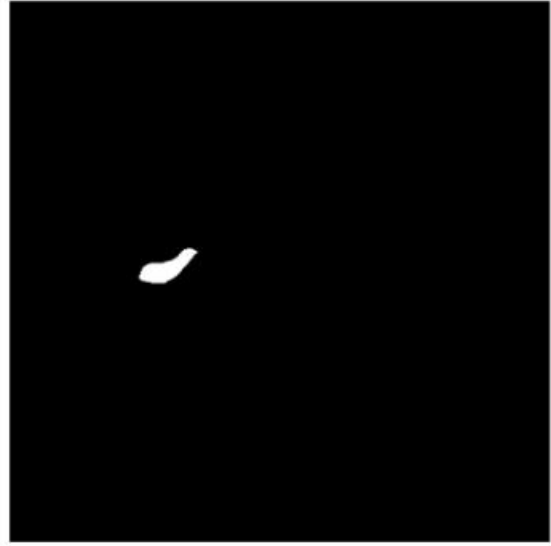


Image: 1060.png

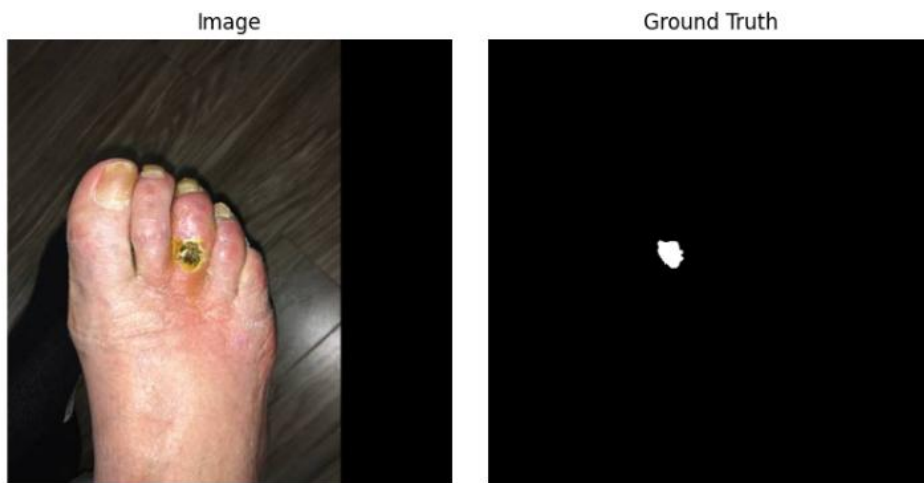


Predicted Mask



```
In [16]: fig, axes = plt.subplots(3, 2, figsize=(8, 12))
for i in range(3):
    img, mask = val_ds[i]
    img_np = img.permute(1,2,0).numpy()
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img_np = (img_np * std + mean).clip(0, 1)
    axes[i][0].imshow(img_np); axes[i][0].set_title("Image"); axes[i][0].axis
    ("off")
    axes[i][1].imshow(mask[0], cmap="gray"); axes[i][1].set_title("Ground Truth"); axes[i][1].axis
    ("off")

plt.tight_layout()
plt.show()
```



```
In [17]: from IPython.display import FileLink

# เช็คว่า save ไว้ที่ไหน
print("Model saved at: ./checkpoints/best_model.pth")

# Download ออกมา
FileLink('./checkpoints/best_model.pth')
```

Model saved at: ./checkpoints/best_model.pth

```
Out[17]: ./checkpoints/best_model.pth
```

New Train

TverskyLoss

New Train

TverskyLoss

In [18]:

```
class FocalTverskyLoss(nn.Module):
    def __init__(self, alpha=0.7, beta=0.3, gamma=0.75, smooth=1e-6):
        super().__init__()
        self.alpha = alpha
        self.beta = beta
        self.gamma = gamma
        self.smooth = smooth

    def forward(self, logits, targets):
        probs = torch.sigmoid(logits)
        tp = (probs * targets).sum(dim=(2, 3))
        fn = ((1 - probs) * targets).sum(dim=(2, 3))
        fp = (probs * (1 - targets)).sum(dim=(2, 3))
        tversky = (tp + self.smooth) / (tp + self.alpha * fn + self.beta * fp + self.smooth)
        return ((1 - tversky) ** self.gamma).mean()

print("FocalTverskyLoss ready ✓")
```

FocalTverskyLoss ready ✓